



Budapesti Komplex Szakképzési Centrum Weiss Manfréd
Technikum, Szakképző Iskola és Kollégium

Vizsgaremek

f1fanclub.hu

Készítette:

- Katona Ruben
- Papp Gergely

Budapest

2026

Tartalomjegyzék

1. Bevezetés

- 1.1. A téma ismertetése és a projekt célja
- 1.2. A témaválasztás indoklása és a piackutatás
- 1.3. Célközönség
- 1.4. Szakmai célkitűzések
- 1.5. Köszönetnyilvánítás
- 1.6. Elérhetőségek

2. Fejlesztői dokumentáció

- 2.1. Fejlesztői környezet és alkalmazott technológiák
 - 2.1.1. Alkalmazott szoftverek és keretrendszerek
 - 2.1.2. Technológiai stack (Backend & Frontend)
 - 2.1.3. Integrált külső könyvtárak és API-k
 - 2.1.4. Fejlesztői munkamegosztás
- 2.2. Adatbázis Modell és Adatszerkezet
 - 2.2.1. Felhasználókezelés és Biztonság (Users & Logs)
 - 2.2.2. Adminisztrációs, biztonsági és preferencia modulok
 - 2.2.3. Közösségi és Paddock Funkciók
 - 2.2.4. Forma-1 statikus adatbázis és a Dinamikus Visszaszámláló
 - 2.2.5. Gamifikáció: „The Pitwall” Típpjáték és Pontbeváltás
 - 2.2.6. Élő Versenyfunkciók és Szimuláció
- 2.3. Biztonsági megoldások a Backendben
- 2.4. Hitelesítési és Jogosultságkezelési Rendszer
- 2.5. Adminisztrációs és Moderációs Rendszer

2.6. Tesztelés leírása

2.7. Technikai továbbfejlesztési lehetőségek

3. Felhasználói dokumentáció

3.1. A program célja és alapvető funkciói

3.2. Szükséges hardvereszközök és szoftverek

3.3. Telepítés és indítás (Elérés)

3.4. A program részletes bemutatása (Szabadon elérhető funkciók)

3.5. Regisztrációhoz kötött funkciók (A közösségi élmény)

3.6. Helytelen használat és hibajelzések

3.7. Információkérés és kapcsolat

4. Összefoglalás

5. Irodalomjegyzék

1. Bevezetés

1.1. A téma ismertetése és a projekt célja

Vizsgaremekünk keretében egy modern és interaktív Forma-1-es rajongói weboldalt és közösségi portált hoztunk létre, amely az „F1 Fan Club” nevet kapta. Projektünk elsődleges célja egy olyan dinamikus platform megalkotása volt, amely szakít a statikus hírportálok hagyományaival. Felületünkön a felhasználók nem csupán a sportággal kapcsolatos legfrissebb adatokat, statisztikákat és versenynaptárakat érhetik el gyorsan, hanem egy aktív közösség részévé is válhatnak a beépített interakciós lehetőségek (posztolás, privát üzenetküldés, tippjáték) révén.

1.2. A témaválasztás indoklása és a piackutatás

Választásunkat személyes érdeklődésünk és egy valós piaci űr felismerése vezérelte. Fejlesztőtársammal elkötelezett rajongókként pontosan tudjuk, milyen funkciókra van szüksége a közösségnek egy versenyhétvége során.

A hazai weben jelenleg is számos F1-es felület működik, azonban ezek túlnyomó többsége egyirányú, klasszikus hírportál. Szükségét éreztük egy olyan közösségépítő platformnak, amely egyszerre nyújt hiteles történelmi adatokat (akár 1950-ig visszamenőleg), és ad teret a rajongói vitáknak. Külön inspirációt jelentett a hivatalos *formula1.com* dizájnjának kritikai elemzése: bár adatgazdag, vizuálisan túlszűfoltnak és nehezen navigálhatónak tartjuk. Ezen tapasztalatokra építve az F1 Fan Club egy letisztult, könnyen áttekinthető, mégis látványos alternatívát kínál.

1.3. Célközönség

Elsődleges célcsoportunk a magyar nyelvű Formula 1-es közösség. A letisztult adatmegjelenítés és a könnyű navigáció az idősebb, statisztikákat kedvelő generáció számára is ideális, míg a beépített közösségi funkciók (Paddock hírfolyam, profil testreszabása, emoji reakciók, jutalomrendszer) kifejezetten a fiatalabb korosztály elvárásaira fókuszálnak.

1.4. Szakmai célkitűzések

A projekt megvalósítása során az alábbi informatikai és fejlesztési célokat tűztük ki:

Dinamikus adatkezelés: Skálázható, biztonságos relációs adatbázis felépítése, valamint külső API-k (pl. Jolpi/Ergast F1) integrálása a historikus adatok megjelenítéséhez.

Modern UI/UX: Egy „Dark Mode” (sötét téma) alapú, prémium érzetet keltő reszponzív felület megalkotása Flexbox és CSS Grid technológiákkal.

Gamifikáció (Játékosítás): A „The Pitwall” tippjáték és a hozzá tartozó pontozási, illetve rangrendszer logikájának kialakítása a felhasználói elköteleződés (retention) növelése érdekében.

Biztonság: A személyes adatok kriptográfiai védelme, szigorú jelszószabályozás és a modern adatbázis-biztonsági eljárások (pl. SQL Injection elleni védelem) implementálása.

1.5. Köszönetnyilvánítás

Ezúton szeretnénk hálás köszönetünket kifejezni felkészítő tanárainknak, **Kovács László** és **Reucov Dávid** tanár uraknak. Szakmai iránymutatásuk, türelmük, és a fejlesztés során nyújtott gyakorlati tanácsaik nélkülözhetetlenek voltak a vizsgaremek sikeres megvalósításához.

1.6. Elérhetőségek

- **Katona Ruben** – katona.ruben@wm-iskola.hu
- **Papp Gergely** – papp.gergely@wm-iskola.hu

2. Fejlesztői dokumentáció

2.1. Fejlesztői környezet és alkalmazott technológiák

A projekt megvalósítása során a modern webfejlesztésben elterjedt, nyílt forráskódú technológiákat és professzionális szerkesztőeszközöket alkalmaztunk. Az alábbiakban összegezzük a fejlesztés alapjait képező szoftvereket és könyvtárakat.

2.1.1. Alkalmazott szoftverek és keretrendszerek

Megnevezés	Funkció / Felhasználási terület	
Visual Studio Code	Elsődleges kódolásra használt környezet.	
XAMPP	Lokális Apache webservert és MySQL adatbázis biztosítása teszteléshez.	
cPanel	Az éles szerver távoli elérése, fájlkezelés és konfiguráció.	
Git / GitHub	Verziókezelés és a forráskód biztonságos, kollaboratív tárolása.	 
phpMyAdmin	Grafikus felület a MySQL adatbázis tervezéséhez és kezeléséhez.	
Windows 11	A fejlesztéshez használt elsődleges operációs rendszer.	

2.1.2. Technológiai stack (Backend & Frontend)

A rendszer magját a PHP és MySQL párosa alkotja, melyet modern kliensoldali megoldásokkal egészítettünk ki:

PHP: Szerveroldali programnyelv, a rendszer üzleti logikájának motorja.

MySQL (MariaDB): Relációs adatbázis-kezelő a felhasználói és statisztikai adatok tárolására.

HTML5 & CSS3: A weboldal strukturális felépítése és reszponzív (mobilbarát) arculata.

JavaScript: Kliensoldali interakciók, dinamikus validációk és aszinkron adatkezelés.



2.1.3. Integrált külső könyvtárak és API-k

Az oldal extra funkcióit (tippjáték, statisztikák, képkezelés) az alábbi speciális megoldásokkal biztosítottuk:

Eszköz / API	Rövid leírás
Jolpi (Ergast) API	Külső adatforrás a történelmi F1-es eredmények és naptárak lekéréséhez.
Sortable.js	A „The Pitwall” tippjáték interaktív Drag & Drop (húzd és ejtsd) mechanikája.
PHPMailer / mail()	Automatizált e-mail küldő rendszer a regisztrációhoz és módosításokhoz.
TinyPNG API	A feltöltött képek automatikus szerveroldali tömörítése és optimalizálása.

2.1.4. Fejlesztői munkamegosztás

A projekt hatékony megvalósítása érdekében a feladatokat a klasszikus kliens-szerver architektúra mentén, két fő területre (Frontend és Backend) osztottuk fel:

Katona Ruben (Backend fejlesztő):

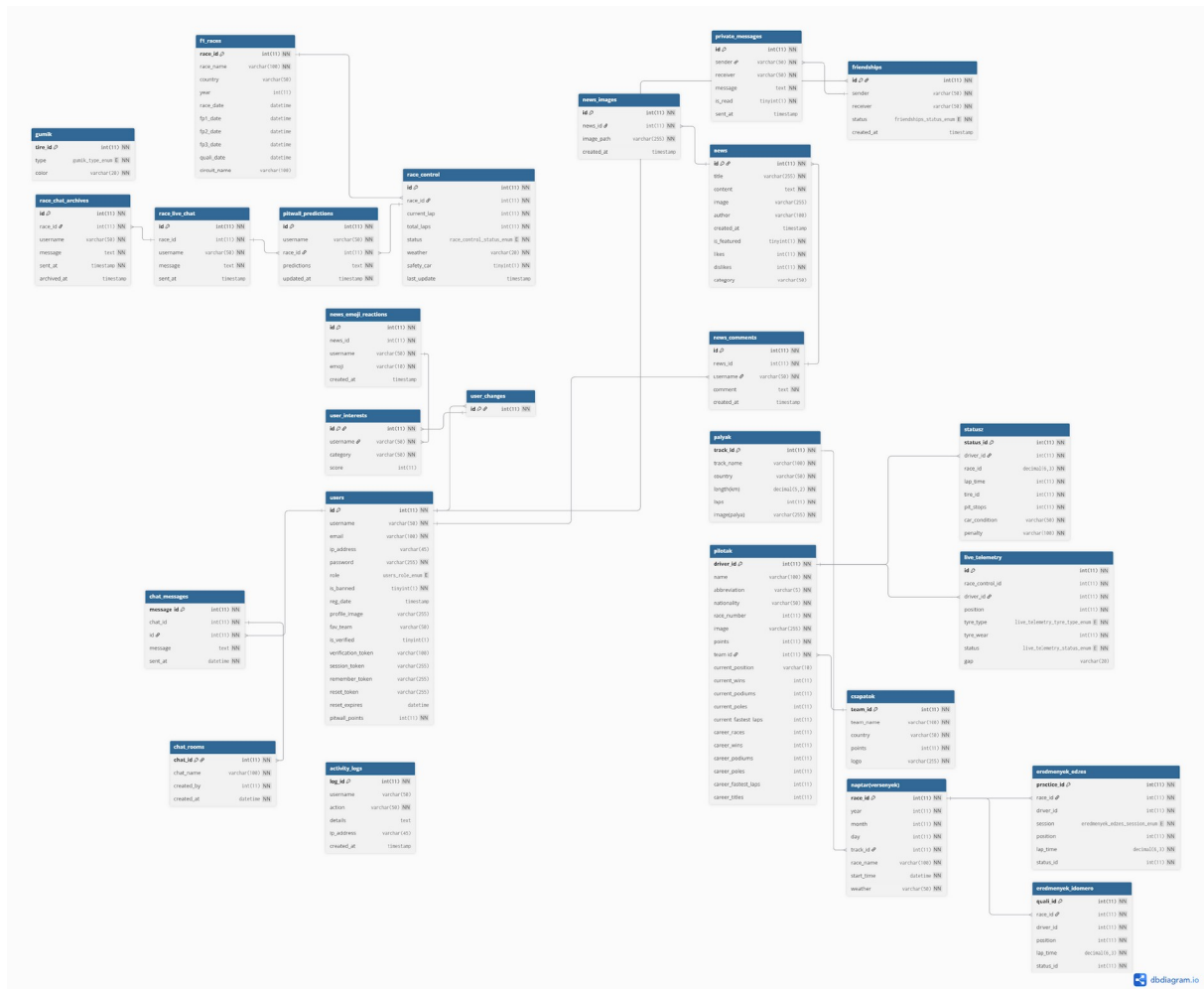
- Adatbázis (MySQL) szerkezetének tervezése és felépítése.
- Szerveroldali PHP API végpontok és az üzleti logika létrehozása.
- Komplex felhasználókezelés (regisztráció, kriptográfia, munkamenet-kezelés).
- Az Adminisztrációs felület és a valós idejű versenyszimuláció fejlesztése.
- A „The Pitwall” tippjáték motorja, automatizált e-mailek és a privát üzenetrendszer programozása.

Papp Gergely (Frontend fejlesztő):

- HTML/CSS struktúra kialakítása és a teljes reszponzív dizájn (mobilbarát nézet) megalkotása.
- Kliensoldali JavaScript interakciók (hamburger menü, lenyíló ablakok, dinamikus galériák) leprogramozása.
- Vizuális effektek, animációk és a dinamikus statisztikai panelek implementálása.
- A böngészők közötti (cross-browser) kompatibilitás biztosítása.
- A Paddock közösségi hírfolyam vizuális felületének (UI) kialakítása.

2.2. Adatbázis Modell és Adatszerkezet:

A rendszer adatbázisa (swmjndga_f1adat) normalizált táblákból épül fel, amelyek funkcionális egységekre bonthatók: Felhasználókezelés, Közösségi funkciók, Forma-1 adatok, Gamifikáció és Élő események.



2.2.1. Felhasználókezelés és Biztonság (Users & Logs)

A users (Felhasználók) tábla Ebben a táblában tároljuk a regisztrált tagok és az adminisztrátorok minden alapvető adatát (username, email). A tábla elsődleges kulcsa az automatikusan növekvő id mező. A rendszer kiemelt figyelmet fordít a kiberbiztonságra és az adminisztrációs kontrollra:

Jelszóvédelem: A jelszavakat sosem nyílt szöveggént, hanem erős kriptográfiai lenyomatként (hash-elve) tároljuk a PHP beépített password_hash() algoritmusával.

Jogosultságkezelés és Moderáció: A role attribútum (ENUM) határozza meg, hogy az adott fiók normál felhasználó (user) vagy adminisztrátor (admin). Az adminisztrátorok által kiosztott esetleges tiltásokat az is_banned logikai (boolean) mező jelöli.

Biztonsági naplózás: A rendszer minden sikeres bejelentkezéskor rögzíti a felhasználó aktuális hálózati azonosítóját az ip_address mezőben.

Tokenek: A session_token és remember_token mezők felelnek a biztonságos munkamenet fenntartásáért. A verification_token az e-mailes fiókaktiválást biztosítja.

Egyéb adatok: A felhasználói élmény személyre szabását a profile_image és fav_team mezők teszik lehetővé. A pitwall_points mező felel a tippjáték virtuális valutájának tárolásáért.

users	
id	int(11) NN
username	varchar(50) NN
email	varchar(100) NN
ip_address	varchar(45)
password	varchar(255) NN
role	users_role_enum E
is_banned	tinyint(1) NN
reg_date	timestamp
profile_image	varchar(255)
fav_team	varchar(50)
is_verified	tinyint(1)
verification_token	varchar(100)
session_token	varchar(255)
remember_token	varchar(255)
reset_token	varchar(255)
reset_expires	datetime
pitwall_points	int(11) NN

2.2.1.2. Adminisztrációs, biztonsági és preferencia modulok

Az alapvető felhasználói adatokon túl a rendszer három speciális adatbázistáblát használ a fiókok védelmére és a felhasználói élmény személyre szabására.

Az **activity_logs** (Tevékenységnapló) tábla:

Ez a tábla a rendszer adminisztrációs és jogi alapköve. Rögzíti a kritikus eseményeket, például a sikeres/sikertelen bejelentkezéseket, fióktiltásokat és jogosultság-módosításokat. Minden rekordhoz eltárolja az akciót végrehajtó felhasználó nevét (username), az esemény típusát (action) és pontos idejét (created_at), valamint a hálózati IP címet (ip_address). Ez az átfogó naplózás elengedhetetlen a rendellenes belépési kísérletek szűréséhez és az esetleges hatósági adatkérések kiszolgálásához.

activity_logs	
log_id	int(11) NN
username	varchar(50)
action	varchar(50) NN
details	text
ip_address	varchar(45)
created_at	timestamp

A **user_changes** (Adatmódosítási kérelmek) tábla:

A biztonsági elvárásoknak megfelelően a felhasználói profiladatok (név, jelszó, e-mail cím) nem változtathatóak meg azonnal. A user_changes tábla egy biztonságos, átmeneti tárolóként funkcionál. Amikor egy tag módosítást kér, a rendszer ide rögzíti az új adatot (new_value), és egy generált, egyedi kriptográfiai tokent (token) küld e-mailben a felhasználónak. A tényleges adatmódosítás a végleges users táblában csak a token aktiválása (az e-mailes megerősítés) után történik meg, így hatékonyan kivédhetőek a fiókfeltörésből származó adatlopások.

#	Név	Típus	Illesztés	Tulajdonságok	Null	Alapértelmezett	Megjegyzések	Extra
☐ 1	id	int(11)			Nem	Nincs		AUTO_INCREMENT
☐ 2	username	varchar(50)	utf8mb4_general_ci		Nem	Nincs		
☐ 3	change_type	varchar(20)	utf8mb4_general_ci		Nem	Nincs		
☐ 4	new_value	varchar(255)	utf8mb4_general_ci		Nem	Nincs		
☐ 5	token	varchar(64)	utf8mb4_general_ci		Nem	Nincs		
☐ 6	created_at	timestamp			Nem	current_timestamp()		

2.2.2. Közösségi és Paddock Funkciók

A közösségi interakciók kezelésére és a felhasználói élmény maximalizálására egy komplex, egymással szorosan összefüggő táblarendszert alakítottunk ki:

news tábla: A Paddock (hírfolyam) motorja. Rögzíti a bejegyzés címét, tartalmát, a szerzőt, a kategóriát, és számolja az interakciókat (reakciók).

news_comments tábla: Minden bejegyzéshez korlátlan számú hozzászólás tartozhat, rögzítve a kommentelőt és a pontos időbélyeget.

news_images tábla: Kapcsolótábla, amely lehetővé teszi, hogy egy poszthoz több kép is tartozzon. Ez a megoldás biztosítja az adatbázis normalizált, redundanciamentes tárolását.

news_emoji_reactions tábla: A modern közösségi média elvárásainak megfelelően a felhasználók emojiakkal reagálhatnak a tartalmakra. A táblában egyedi (UNIQUE) kulcsot állítottunk be a news_id, username és emoji hármására, adatbázis szinten megakadályozva a spammelést.

Bemutatott táblák:

news	
id	int(11) NN
title	varchar(255) NN
content	text NN
image	varchar(255)
author	varchar(100)
created_at	timestamp
is_featured	tinyint(1) NN
likes	int(11) NN
dislikes	int(11) NN
category	varchar(50)

news_comments	
id	int(11) NN
news_id	int(11) NN
username	varchar(50) NN
comment	text NN
created_at	timestamp

news_images	
id	int(11) NN
news_id	int(11) NN
image_path	varchar(255) NN
created_at	timestamp

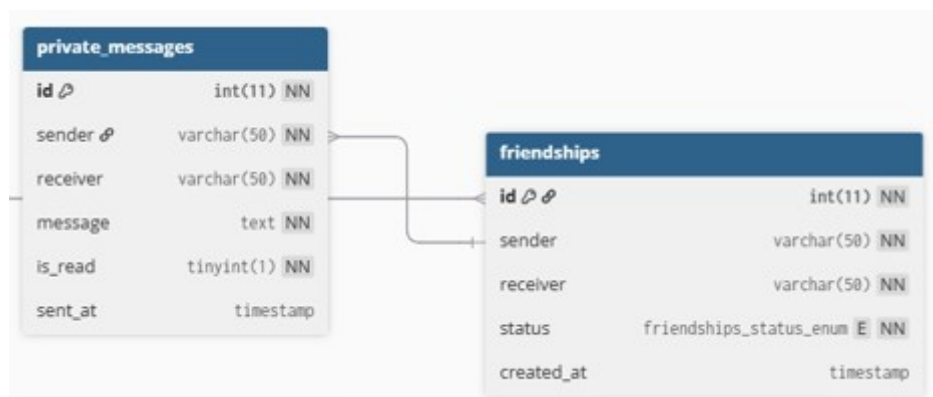
user_interests tábla: A Paddock hírfolyam intelligens, személyre szabott algoritmusának alapját képezi. Rögzíti, hogy a felhasználó mely kategóriájú bejegyzésekkel lép leggyakrabban interakcióba, így a hírfolyam betöltésekor a rendszer ezen preferenciák alapján súlyozza és rendezi a legrelevánsabb posztokat.

id	username	category	score
1	rubenA	Drivers	112
2	rubenA	Red Bull	13
3	HELLJUMPER	Drivers	23

user_interests	
id	int(11) NN
username	varchar(50) NN
category	varchar(50) NN
score	int(11)

friendships tábla: A tagok közötti baráti kapcsolatokat tárolja és adminisztrálja (pending, accepted státuszokkal).

private_messages tábla: A valós idejű privát kommunikációért felel a barátok között. Az is_read logikai mező segítségével egy hatékony értesítési (notification) rendszert alakítottunk ki a frontend oldalon az olvasatlan üzenetek jelzésére.



A csapatok és pilóták táblák

Ezek a relációs táblák a 2026-os szezon aktuális felállítását és alapvető statisztikáit tartalmazzák. A két tábla között klasszikus egy-a-többhöz (1:N) kapcsolat áll fenn: a pilóták rekordjai egy team_id idegen kulccsal (Foreign Key) hivatkoznak a saját csapatukra. Ez az adatbázis-szintű összekapcsolás teszi lehetővé, hogy a frontend oldalon a pilóták adatlapjai, valamint a felhasználói profilok dinamikusan, mindig az adott csapat hivatalos arculati színeiben (pl. Ferrari piros, McLaren papaya) jelenjenek meg, elkerülve a redundáns adattárolást.

pilóták	
driver_id 🔗	int(11) NN
name	varchar(100) NN
abbreviation	varchar(5) NN
nationality	varchar(50) NN
race_number	int(11) NN
image	varchar(255) NN
points	int(11) NN
team_id 🔗	int(11) NN
current_position	varchar(10)
current_wins	int(11)
current_podiums	int(11)
current_poles	int(11)
current_fastest_laps	int(11)
career_races	int(11)
career_wins	int(11)
career_podiums	int(11)
career_poles	int(11)
career_fastest_laps	int(11)
career_titles	int(11)

csapatok	
team_id 🔗	int(11) NN
team_name	varchar(100) NN
country	varchar(50) NN
points	int(11) NN
logo	varchar(255) NN

f1_races (Versenynaptár) tábla és a Dinamikus Visszaszámláló

Ez a tábla a rendszer központi „időzítő motorja”. A versenynaptár minden eseményét pontos DATETIME formátumban tárolja. Ennek a táblának kritikus, kettős funkciója van: egyrészt ez határozza meg a „The Pitwall” tippjáték backend logikáját (a tippelés lezárását a rajt pillanatában), másrészt ez vezérli a főoldalon (index.php) talá lható dinamikus visszaszámlálót.

A visszaszámláló modul (Countdown) megvalósítása egy optimalizált, hibrid kliens-szerver architektúrára épül, amely az alábbi lépésekben működik:

f1_races	
race_id	int(11) NN
race_name	varchar(100) NN
country	varchar(50)
year	int(11)
race_date	datetime
fp1_date	datetime
fp2_date	datetime
fp3_date	datetime
quali_date	datetime
circuit_name	varchar(100)

Szerveroldali adatkérés (PHP & MySQL): Amikor a felhasználó betölti a főoldalt, a PHP egy optimalizált lekérdezést futtat (SELECT * FROM f1_races WHERE race_date > NOW() ORDER BY race_date ASC LIMIT 1). A lekérdezés szűri a múltbéli eseményeket, és kizárólag a legközelebbi, még le nem zajlott futam adatait kéri le a szervertől.

```
69 $sqlNextRace = "SELECT * FROM f1_races WHERE race_date > NOW() ORDER BY race_date ASC LIMIT 1";
70 $resultRace = $conn->query($sqlNextRace);
71
```

Adatátadás a kliensnek: A PHP a kapott pontos dátumot átalakítja egy JavaScript által értelmezhető formátummá, majd a weboldal forráskódjába generálja, mint statikus változót.

Kliensoldali aszinkron frissítés (JavaScript): A kapott időbélyeget (timestamp) egy kliensoldali JavaScript függvény (updateCountdown()) veszi át. A böngészőben futó szkript a setInterval() metódus segítségével folyamatosan kiszámítja a különbséget a célidőpont és a felhasználó eszközének aktuális ideje között. Az eredményt napokra, órákra és percekre bontja, majd a DOM (Document Object Model) manipulációjával frissíti a képernyőn látható értékeket anélkül, hogy a teljes weboldalt újra kellene tölteni. Ezzel jelentős szerver-erőforrást spórolunk meg.

```
835 <?php if ($nextRace): ?>
836 const raceDateStr = "<?php echo date('Y-m-d H:i:s', strtotime(\$nextRace['race_date'])); ?>";
837 const raceDate = new Date(raceDateStr).getTime();
838
839 ~ function updateCountdown() {
840     const now = new Date().getTime();
841     const timeLeft = raceDate - now;
842
843 ~     if (timeLeft > 0) {
844         const days = Math.floor(timeLeft / (1000 * 60 * 60 * 24));
845         const hours = Math.floor((timeLeft % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60));
846         const minutes = Math.floor((timeLeft % (1000 * 60 * 60)) / (1000 * 60));
847
848         document.getElementById('days').textContent = days.toString().padStart(2, '0');
849         document.getElementById('hours').textContent = hours.toString().padStart(2, '0');
850         document.getElementById('minutes').textContent = minutes.toString().padStart(2, '0');
851 ~     } else {
852         const timerContainer = document.querySelector('.countdown-timer');
853 ~         if (timerContainer) {
854             timerContainer.innerHTML = '<div class="countdown-unit"><span class="countdown-value" style="font-size:1.5rem;">VERSENYHÉTVÉGE!</span></div>';
855         }
856     }
857
858     updateCountdown();
859     setInterval(updateCountdown, 60000);
860 <?php endif; ?>
```

Eseménykezelés lejáratkor: A szkript logikája tartalmaz egy elágazást is: amennyiben a hátralévő idő eléri a nullát, a JavaScript azonnal beavatkozik a HTML szerkezetbe, és a számokat tartalmazó blokkokat lecseréli egy központi, „VERSENYHÉTVÉGE!” feliratot tartalmazó elemre.

2.2.3. Élő Versenyfunkciók és Szimuláció (Live Race Engine)

A weboldal technológiailag legösszetettebb modulja a versenyhétvégék alatti élő szimuláció, amely valós idejű (real-time) adatszinkronizációt igényel a szerver és a kliensek között. A rendszert a maximális teljesítményre és az adatbázis tehermentesítésére hangolva terveztük meg, amelyet egy API-vezérelt (Application Programming Interface) hibrid architektúra szolgál ki.

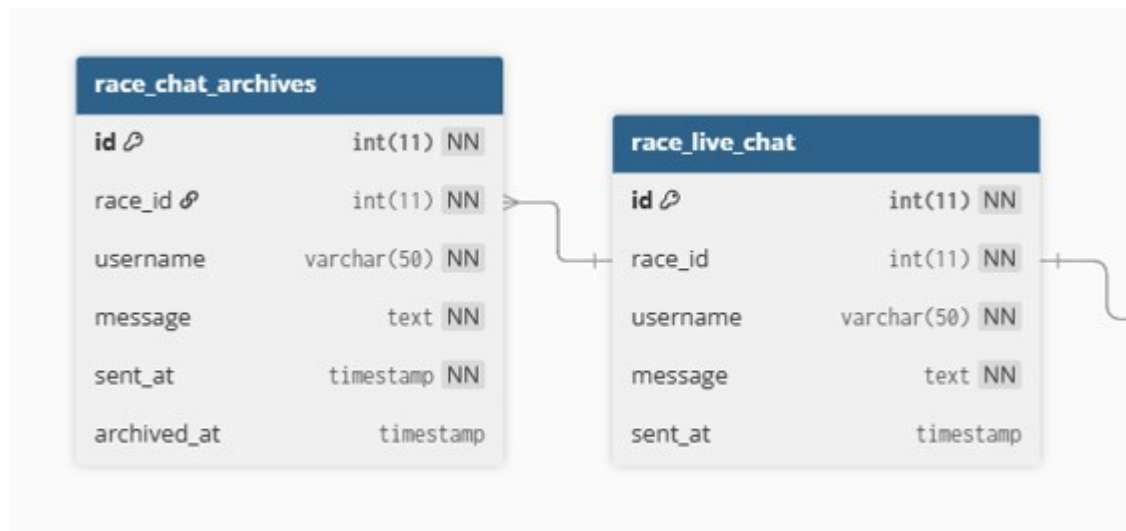
I.Szerveroldali Állapotgép és Telemetria

A szimuláció adatbázis-szintű vezérlését két fő tábla látja el:

race_control tábla: Ez funkcionál a rendszer szerveroldali "Állapotgépeként" (State Machine). Tárolja a futam globális változóit: az aktuális körszámot, a biztonsági autó (Safety Car) jelenlétét, az időjárást, valamint a futam státuszát (running, stopped, vagy archived). Ez a tábla működik "biztonsági kapuként" is: ha a státusz archived (befejezett), a live.php oldal azonnal, szerveroldalon (PHP header redirect) visszairányítja a felhasználót a főoldalra, megakadályozva a lezárt futamokhoz való jogosulatlan hozzáférést.

race_control		
id		int(11) NN
race_id		int(11) NN
current_lap		int(11) NN
total_laps		int(11) NN
status	race_control_status_enum E	NN
weather		varchar(20) NN
safety_car		tinyint(1) NN
last_update		timestamp

live_telemetry tábla: Rögzíti a pilóták pillanatnyi pozícióját, a közöttük lévő időkülönbséget (Gap), a gumiválasztást és a kieséseket (DNF). A race_api.php végpont ezt a táblát egy összetett SQL relációval (JOIN) kapcsolja össze a pilóták és csapatok táblákkal. Így a frontend egyetlen, tiszta JSON adatcsomagban kapja meg a pilóta nevét, a csapat logóját és a telemetria adatokat, minimalizálva a szerverhez intézett kérések számát.



II. Valós idejű aszinkron adatlekérés (AJAX Polling)

Mivel a rendszer hagyományos PHP/Apache környezetben fut (ahol a WebSocket technológia natívan nehezen implementálható), az adatszinkronizációt egy úgynevezett **Short Polling** eljárással oldottuk meg. A live.php kliensoldali JavaScript kódja a setInterval() metódus segítségével 3 másodpercenként aszinkron kéréseket (Fetch API) küld a race_api.php és a chat_api.php végpontoknak. A kapott JSON válaszok alapján a böngésző csak a szükséges DOM (Document Object Model) elemeket frissíti, így a teljes weboldal újratöltése nélkül is folyamatos, folyékony élményt biztosít a felhasználónak.

```

1379 ▾ async function sendChat() {
1380     const input = document.getElementById('chatInputMsg');
1381     if(!input || input.value.trim() === '') return;
1382
1383     let msg = input.value;
1384     input.value = '';
1385
1386 ▾     await fetch('chat_api.php', {
1387         method: 'POST',
1388         headers: {'Content-Type': 'application/json'},
1389         body: JSON.stringify({ message: msg })
1390     });
1391     loadChat();
1392 }
1393
1394 fetchData();
1395 loadChat();
1396 setInterval(loadChat, 3000);
1397
1398 ▾ window.addEventListener('resize', function() {
1399 ▾     if (window.lastRaceData) {
1400         renderRace(window.lastRaceData);
1401     }
1402 });
  
```

III. Élő Chat és Archiválási Logika

Egy élő verseny alatt a chat funkció másodpercenként akár tucatnyi adatbázis-írást és olvasást is generálhat. A rendszer összeomlásának elkerülése érdekében egy speciális adatbázis-menedzsment logikát vezettünk be:

```
1 <?php
2 session_start();
3 header('Content-Type: application/json');
4 header("Cache-Control: no-store, no-cache, must-revalidate, max-age=0");
5 header("Cache-Control: post-check=0, pre-check=0", false);
6 header("Pragma: no-cache");
```

Aktív vs. Hideg tárolás: A rendszer külön kezeli az élő "tranzakciókat"

(race_live_chat tábla) és a múltbéli adatokat (race_chat_archives). Az élő táblában mindig csak az aktuális futam üzenetei szerepelnek. A futam leintését követően egy adminisztrátori parancs az összes üzenetet átmozgatja az archív táblába ("hideg tárolás"). Ezzel az eljárással az élő tábla mérete kicsi, az indexelése és az olvasási sebessége pedig villámgyors marad a következő futamra.

Adatintegritás (LEFT JOIN): A chat_api.php az üzenetek lekérésekor a users táblával egy LEFT JOIN kapcsolatot használ. Ez a technikai döntés garantálja, hogy ha egy felhasználó időközben nevet változtat vagy törli a profilját, a korábban elküldött chat üzenetei nem tűnnek el a rendszerből, ezáltal nem sérül a beszélgetés kontextusa.

```
43 $sql = "SELECT c.id, c.username, c.message, c.sent_at, u.profile_image, u.fav_team
44         FROM race_live_chat c
45         LEFT JOIN users u ON c.username = u.username
46         WHERE c.race_id = 25
47         ORDER BY c.id ASC";
```

Gyorsítótár tiltása: A chat zavartalan működése érdekében a PHP fejlécekben (Headers) szigorú szabályokat (Cache-Control: no-store, no-cache) állítottunk be, kényszerítve a böngészőt, hogy a chat frissítésekor soha ne a memóriából (cache), hanem mindig a szerverről kérje le a legfrissebb adatokat.

2.2.4. Gamifikáció: „The Pitwall” Típpjáték és Pontbeváltás

A felhasználói elköteleződés és a visszatérési arány (retention) növelése érdekében a portál egy beépített típpjátékkal bővült. A játékosok a futamok előtt megtippelhetik a pilóták beérkezési sorrendjét, a sikeres találatokért pedig virtuális valutát (pitwall_points) kapnak, amelyet a beépített webshopban válthatnak be.

I. Kliensoldali működés és Határidő-kezelés (pitwall.php)

A tippelő felület vizuális alapját a **Sortable.js** nyílt forráskódú könyvtár adja, amely lehetővé teszi a pilótákat tartalmazó HTML listaelemek zökkenőmentes, vizuális átrendezését (Drag & Drop). Az adatintegritás megőrzése érdekében a PHP backend szigorú határidő-ellenőrzést végez:

```
52 $raceTimestamp = strtotime($nextRace['race_date']);
53 $deadlineTimestamp = $raceTimestamp - 86400;
54 $deadlineStr = date('Y. m. d. H:i', $deadlineTimestamp);
55
```

A kód kiszámolja a futam kezdetét, és abból pontosan 24 órát (86400 másodpercet) levonva határozza meg a határidőt. Amint a szerveridő átlépi ezt a pontot, a felület "szürkévé" válik, a mentés gomb pedig letiltásra (disabled) kerül, megakadályozva a futam közbeni csalásokat. II. Adatbázis optimalizáció: JSON alapú mentés (pitwall_api.php)

A klasszikus relációs adatbázis-tervezésnél a 20 pilóta helyezésének tárolása 20 különálló oszlopot vagy egy bonyolult kapcsolótáblát igényelne. A lekérdezések sebességének maximalizálása érdekében ehelyett egy modern **JSON alapú adattárolást** alkalmaztunk a pitwall_predictions táblában.

Amikor a felhasználó a mentés gombra kattint, a JavaScript egy tömbbe gyűjti az aktuális sorrendet, és a Fetch API segítségével aszinkron módon elküldi a pitwall_api.php végpontnak. A szerver a json_encode() függvénnyel a teljes listát egyetlen TEXT formátumú stringgá alakítja (pl.: ["4","1","16","44",...]), majd menti. A mentés során egy haladó SQL megoldást, az **Upsert (ON DUPLICATE KEY UPDATE)** parancsot alkalmaztuk:

```
32 $sql = "INSERT INTO pitwall_predictions (username, race_id, predictions)
33     VALUES (?, ?, ?)
34     ON DUPLICATE KEY UPDATE predictions = VALUES(predictions)";
```

Ez a logika egyetlen lekérdezésben dönti el, hogy a felhasználó most tippel először (ekkor létrehozza a rekordot), vagy egy meglévő tippjét módosítja (ekkor felülírja azt).

III. A Kiértékelés (Hit Detection) logikája

A futam hivatalos leintése után a rendszer kiértékeli a leadott tippeket. Mivel a tippeket és a valós végeredményt is egy-egy strukturált JSON tömbként kezeljük, a kiértékelő algoritmus egy egyszerű iterációval (ciklussal) halad végig az indexeken. A rendszer összehasonlítja, hogy a felhasználó tömbjének X. indexén álló pilóta ID-ja megegyezik-e a valós eredmény X. indexével. Minden pontos egyezés után a rendszer 10 pontot ír jóvá a users tábla pitwall_points mezőjében.

```
24 $prediction = json_decode($user_row['predictions'], true);
25 $earned_points = 0;
26
27 foreach($prediction as $index => $predicted_driver_id) {
28     if(isset($real_results[$index]) && $predicted_driver_id == $real_results[$index]) {
29         $earned_points += 10;
30     }
31 }
```

IV. A Shop és a Beváltási folyamat (bevaltas.php)

Az összegyűjtött pontokat a játékosok a The Pitwall Shop felületén költhetik el VIP rangokra, kuponokra vagy nyereményjáték belépőkre. A vásárlási logika kiemelten figyeli a pont-egyenleget és az e-mail kommunikációt:

Validáció és Tranzakció: A PHP első lépésben ellenőrzi, hogy a felhasználó egyenlege nagyobb vagy egyenlő-e a kiválasztott nyeremény árával. Ha igen, azonnal levonja a pontokat az adatbázisból.

Automatikus E-mail küldés: Sikeres tranzakció esetén a backend a PHP beépített mail() függvényét használva automatikusan generál egy visszaigazoló e-mailt a felhasználónak. A levél tartalmazza a beváltott nyeremény pontos megnevezését (pl. 10% F1 Webshop Kupon), a fennmaradó pontegyenleget, valamint tájékoztatást a további teendőkről. A felület a kliensoldalon egy vizuális siker-üzenettel (Alert) nyugtázza a folyamatot, bezárva ezzel a gamifikációs körforgást.

```
58 if ($userPoints >= $cost) {
59
60     $newPoints = $userPoints - $cost;
61     $updateStmt = $conn->prepare("UPDATE users SET pitwall_points = ? WHERE username = ?");
62     $updateStmt->bind_param("is", $newPoints, $username);
63
64     if ($updateStmt->execute()) {
65         $userPoints = $newPoints;
66
67
68         $to = $userEmail;
69         $subject = "Sikeres beváltás: $title - F1 Fan Club";
70         $emailBody = "Kedves $username!\n\n"
71             . "Gratulálunk! Sikeresen beváltottál $cost Pitwall pontot a következő nyereményre:\n\n"
72             . "--- $title ---\n\n"
73             . "A nyereményeddel (vagy a kuponkóddal) kapcsolatban az adminisztrátoraink  

74             hamarosan felveszik veled a kapcsolatot ezen az e-mail címen.\n\n"
75             . "Aktuális pontegyenleged: $userPoints PONT.\n\n"
76             . "Üdvözlettel,\nAz F1 Fan Club Csapata";
```

2.3. Hitelesítési és Jogosultságkezelési Rendszer (Authentication)

A register.php, verify.php és login.php fájlok által vezérelt hitelesítési modul felel a felhasználók biztonságos kezeléséért, a legszigorúbb biztonsági elvárások betartásával.

2.3.1. Regisztrációs folyamat és Validáció

Az űrlap beküldésekor a backend a következő logikai lépéseken halad végig:

- **Adattisztítás és szűrés:** A bejövő adatokat a trim() függvénnyel megtisztítjuk, majd egy adatbázis-lekérdezéssel kiszűrjük a redundáns (már létező) felhasználóneveket és e-mail címeket.
- **Token generálás és mentés:** Érvényes adatok esetén a rendszer a bin2hex(random_bytes(32)) algoritmussal egy 64 karakteres, kriptográfiailag biztonságos aktivációs tokent generál. A fiók ekkor még inaktív (is_verified = 0) státusszal kerül az adatbázisba.

	id	username	verification_token
☐ Módosítás Másolás Törlés	40	proba1	4c522ed93404deb61e3fb5f45e10a6088ec05a69f6aa3cec5d...

```
15 $message = "";
16 $messageType = "error";
17
18 if (isset($_GET['token'])) {
19     $token = trim($_GET['token']);
20
21     $stmt = $conn->prepare("SELECT id, username FROM users WHERE verification_token = ? AND is_verified = 0");
22     $stmt->bind_param("s", $token);
23     $stmt->execute();
24     $result = $stmt->get_result();
25
26     if ($result->num_rows > 0) {
27         $user = $result->fetch_assoc();
28         $userId = $user['id'];
29         $username = $user['username'];
30
31         $stmt_update = $conn->prepare("UPDATE users SET is_verified = 1, verification_token = NULL WHERE id = ?");
32         $stmt_update->bind_param("i", $userId);
33
34         if ($stmt_update->execute()) {
35             $message = "Szia <strong>$username</strong>!<br>A fiókotat sikeresen aktiváltuk!";
36             $messageType = "success";
37         } else {
38             $message = "Hiba történt az aktiválás közben: " . $conn->error;
39         }
40         $stmt_update->close();
41
42     } else {
43         $message = "Ez az aktiváló link érvénytelen, vagy a fiókotat már korábban aktiváltad.";
44     }
45     $stmt->close();
46 } else {
47     $message = "Hiányzó aktiváló kód!";
48 }
49 $conn->close();
50 ?>
```

2.3.2. E-mailes megerősítés (Double Opt-in)

Az inaktív fiókkal nem lehet bejelentkezni az oldalra. A regisztráció során generált egyedi tokent a rendszer a PHP beépített mail() függvénye segítségével (biztonságos noreply címről) elküldi a felhasználónak egy aktivációs link formájában (pl. verify.php?token=...). Amikor a felhasználó a linkre kattint, a rendszer ellenőrzi a tokent az adatbázisban. Egyezés esetén a fiók aktiválódik (UPDATE users SET is_verified = 1), az egyszer használatos token pedig azonnal megsemmisül (verification_token = NULL), így védve a rendszert a Replay-jellegű támadások ellen.

2.3.3. Bejelentkezés és Tevékenységnaplózás

A bejelentkezési kísérlet során a felhasználónév és a jelszó validálása a password_verify() beépített PHP függvénnyel történik, amely biztonságosan hasonlítja össze a nyílt szöveget az adatbázisban lévő hash-sel. Sikeres hitelesítés esetén a szerver komplex adminisztrációs feladatokat hajt végre a háttérben:

```
23  if (password_verify($password, $user['password'])) {  
24  if ($user['is_verified'] == 0) {  
196  
197      session_regenerate_id(true);  
198      $_SESSION['username'] = $user['username'];  
199      $_SESSION['role'] = $user['role'];  
200  
201      $ip_address = $_SERVER['REMOTE_ADDR'];  
202      $update_ip = $conn->prepare("UPDATE users SET ip_address = ? WHERE id = ?");  
203      $update_ip->bind_param("si", $ip_address, $user['id']);  
204      $update_ip->execute();  
205      $update_ip->close();
```

2.3.4. Jogosultságvizsgálat és Tartós bejelentkezés (Remember Me)

A bejelentkezés utolsó szakaszában a rendszer ellenőrzi a felhasználó jogosultsági szintjét (role). Amennyiben a role értéke admin, a felhasználó az adminisztrációs vezérlőpult (Admin Panel) kerül átirányításra, míg a normál tagok a kezdőlapot érik

el. Amennyiben a felhasználó a "Emlékezz rám" (Remember me) opciót választotta, a rendszer egy újabb, 32 bájtos kriptográfiai tokent generál, amelyet összekapcsol az adatbázis rekorddal, majd a kliensoldalon egy 30 napos lejáratú süti (Cookie) tárolja el a `setcookie()` függvény segítségével. Ezzel a megoldással kényelmes, mégis biztonságos tartós belépést biztosítunk a felhasználóknak.

```
219 ▾ if (isset($_POST['remember_me'])) {
220     $token = bin2hex(random_bytes(32));
221     setcookie('remember_me', $token, time() + (86400 * 30), "/");
222     $update_token = $conn->prepare("UPDATE users SET remember_token = ? WHERE id = ?");
223     $update_token->bind_param("si", $token, $user['id']);
224     $update_token->execute();
225 }
226
227 ▾ if ($user['role'] === 'admin') {
228     header("Location: /f1fanclub/admin/admin.php");
229 ▾ } else {
230     header("Location: /f1fanclub/index.php");
231 }
232 exit;
```

Bejelentkezés a bemutatóhoz:

Felhasználónév: admin_felh

Jelszó: Admin_felh12@

Szerepkör: admin

2.4. Biztonsági Megoldások a Backendben

Mivel az F1 Fan Club egy publikus, felhasználói adatokat és interakciókat kezelő platform, a tervezés során kiemelt prioritást kapott az adatszivárgások és

kiberbiztonsági fenyegetések elleni védekezés. A projekt során a legmodernebb iparági webes biztonsági szabványokat alkalmaztuk:

1. **SQL Injection (SQLi) elleni védelem:** A felhasználótól érkező nyers adatokat (pl. űrlap bemenetek, URL paraméterek) sosem fűzzük közvetlenül az SQL parancsokhoz. Minden adatbázis-műveletet a MySQLi kiterjesztés Előkészített Utasítások (Prepared Statements) és bind_param módszerével hajtunk végre. Ez az eljárás szétválasztja az SQL logikát az adatoktól, így lehetetlenné teszi a kártékony SQL kódok befecskendezését.
2. **Jelszó komplexitás és Kriptográfia:** A regisztráció során szigorú szerveroldali Reguláris Kifejezések (Regex) biztosítják a megfelelő jelszóerősséget. Az ellenőrző minta: `^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[a-zA-Z\d\w\W]{8,}$`. Ez garantálja, hogy a jelszó minimum 8 karakter hosszú legyen, és tartalmazzon kisbetűt, nagybetűt, valamint számot. A jelszavakat nyílt szöveggént sosem tároljuk, kizárólag a PHP beépített `password_hash()` függvényével generált, egyirányú kriptográfiai lenyomat formájában kerülnek az adatbázisba.

```
16 if ($_SERVER["REQUEST_METHOD"] == "POST") {  
17     $username = trim($_POST['username']);  
18     $email = trim($_POST['email']);  
19     $password = $_POST['password'];  
20  
21     if (!preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[a-zA-Z\d\w\W]{8,}$/', $password)) {  
22         ?>
```

3. **Tranzakció kezelés (Rollback) és Adatintegritás:** Kritikus adatmódosításoknál (például egy felhasználónév megváltoztatásakor, ahol egyszerre több táblában kell átírni a relációs adatokat) az SQL BEGIN TRANSACTION és COMMIT logikát alkalmazzuk. Bármilyen futásidejű hiba esetén a ROLLBACK parancs azonnal visszaállítja az adatbázist az eredeti, módosítás előtti állapotába, megakadályozva a részleges és hibás adatszinkronizációt.

2.5. Szoftvertesztelés és Minőségbiztosítás

A webalkalmazás tesztelése nem egy utólagos fázis volt, hanem a fejlesztés teljes életciklusa során, az agilis módszertanokhoz hasonlóan, iteratívan zajlott. Bár a backend logikát kód szinten ellenőriztük, a frontend tesztelése manuálisan (Black-

box tesztelés) történt, mivel a felhasználói felület (UI) vizuális ellenőrzése és a felhasználói élmény (UX) szubjektív értékelése kiemelt fontosságú volt.

2.5.1. Reszponzivitás és böngésző-kompatibilitás

A modern weben alapelvárás a mobil-első (Mobile-first) megközelítés. Az oldal rezponzív viselkedését a Chrome DevTools eszköztárával, valamint valós, fizikai eszközökön teszteltük, hogy garantáljuk a böngészők közötti (Cross-browser) kompatibilitást.

Tesztelt Eszköz / Felbontás	Használt Böngésző(k)	Teszt Eredménye
Asztali PC (1920x1080)	Chrome, Firefox, Edge, Safari	Hibátlan (Natív nézet)
Laptop (1366x768)	Chrome, Firefox	Megfelelő (Kompakt asztali nézet)
iPad Air (820x1180)	Safari, Chrome	Megfelelő (Tablet töréspont)
iPhone 14 Pro (390x844)	Safari, Chrome	Megfelelő (Mobil nézet, hamburger menü)
Samsung Galaxy S20	Chrome, Samsung Internet	Működik (Android specifikus renderelés)

Minden eszközön kiemelten ellenőriztük a kritikus interakciós pontokat: a hamburger menü állapotváltozásait, a CSS Grid alapú galériák érintőképernyős (Swipe) görgetését, a Paddock text-area magatartását virtuális billentyűzet esetén, valamint a „The Pitwall” felület Drag & Drop funkciójának működését mobil eszközökön.

2.5.2. Funkcionális tesztelés (Kivonat)

A rendszer üzleti logikájának validálására specifikus teszteseteket (Test Cases) írtunk fel. Az alábbi táblázatok a legkritikusabb modulok tesztelési eredményeit mutatják be.

I. Hitelesítés és Felhasználókezelés

Teszteteset leírása	Várt Eredmény (Expected)	Státusz
Regisztráció már létező felhasználónévvel/emaillel	SQL hiba helyett formázott frontend hibaüzenet	Sikeres
Regisztráció gyenge jelszóval (Regex teszt)	Frontend blokkolás és hibaablak megjelenítése	Sikeres
Fiókaktiválás lejárt/hamis tokennel	Adatbázis elutasítja, hibaüzenet: "Érvénytelen link"	Sikeres
"Emlékezz rám" (Remember me) funkció	30 napos süti beállítása, böngésző bezárása után is aktív session	Sikeres

II. Paddock (Közösségi hírfolyam) és Chat

Teszteteset leírása	Várt Eredmény (Expected)	Státusz
Poszt létrehozása kötelező kategória nélkül	A "Posztolás" gomb inaktív (disabled) marad a DOM-ban	Sikeres
Képfeltöltés 5MB feletti fájl mérettel	Kliensoldali blokkolás, szerver nem terhelődik	Sikeres
Emoji reakció hozzáadása és visszavonása	A JavaScript aszinkron frissíti az adatbázist és a UI számlálót	Sikeres
Élő chat automatikus frissítése	3 másodpercenkénti AJAX kérés, új üzenetnél görgetés az aljára	Sikeres

III. Gamifikáció és Hibakezelés

Teszteteset leírása	Várt Eredmény (Expected)	Státusz
Tippelés módosítása a The Pitwallon	Adatbázis oldali "Upsert" felülírja a korábbi JSON tömböt	Sikeres
Tippelés beküldése a 24 órás határidő lejártá után	A frontend gomb eltűnik, a backend visszautasítja a kérést	Sikeres
Jutalom beváltása elégtelen egyenleggel	Tranzakció blokkolása: "Nincs elég pontod" üzenet	Sikeres

Teszteteset leírása	Várt Eredmény (Expected)	Státusz
Nem létező oldal közvetlen elérése (URL manipuláció)	Szerveroldali 404-es hiba vagy átirányítás a főoldalra	Sikeres

2.6. Frontend Architektúra és Felhasználói Élmény (UI/UX)

A weboldal kliensoldali felülete (Frontend) natív HTML5, CSS3 és JavaScript (ES6) technológiákkal készült. Tudatos mérnöki döntés volt a nehéz keretrendszerek (pl. React, Vue) mellőzése. Ennek oka egyrészt a teljesítmény-optimalizálás (kevesebb betöltendő JavaScript fájl), másrészt a tanulási folyamat maximalizálása volt: célunk a DOM manipuláció, az aszinkron adatkérések és a CSS vizuális motorjának mély szintű megértése volt.

2.6.1. A sötét téma (Dark Mode) pszichológiája és tervezése

Az oldal alapértelmezett megjelenése egy teljes egészében sötét téma (Dark Mode), amely a Forma-1 prémium, high-tech hangulatát idézi. A sötét felület melletti döntésünket az alábbi UI/UX kutatási elvek indokolták:

1. **Szemfáradás csökkentése:** A Forma-1-es futamok a magyar időzónában gyakran kora hajnalban vagy késő este zajlanak. Egy vakítóan fehér háttérű weboldal sötét környezetben kifejezetten fárasztó a felhasználó szemének.
2. **Vizuális kontraszt és Brand identitás:** Sötét háttéren (Mély fekete: #0a0a0a) a csapatok egyedi, élénk színei (pl. Ferrari piros #DC0000, Mercedes türkiz #00D2BE) sokkal intenzívebben, szinte "izzó" hatással (Glow effect) jelennek meg.
3. **Dizájn-kényszer:** Egy világos téma könnyen elfedi a tervezési hiányosságokat, míg a sötét téma kíméletlenül megmutatja a kontraszthibákat. Ez a tudatos döntés folyamatosan magas minőségű, átgondolt vizuális hierarchia kialakítására ösztönzött minket.

2.6.2. Reszponzív elrendezés: Flexbox és CSS Grid

A weboldal struktúráját a CSS Flexbox és Grid rendszerek hibrid használatával építettük fel. A Flexbox felel a fejléc, a navigáció és a dinamikus kártyák egydimenziós (sor vagy oszlop) elrendezéséért, míg a CSS Grid a statisztikai paneleknél és a képgalériáknál biztosítja a precíz, kétdimenziós struktúrát.

A részponzivitást CSS Media Query-k segítségével (@media) öt fő törésponton (Breakpoint) finomhangoltuk:

- **1200px:** Kisebb asztali monitorok (a kártyák 3-ról 2 oszlopra váltanak).
- **992px:** Tablet váltás (a hagyományos navigációt felváltja a JavaScript-vezérelt Hamburger menü).
- **768px:** Mobil nézet (a galériák vízszintes görgetése függőleges listává alakul, a hover effekteket felváltják az érintés-alapú események).
- **576px és 360px:** Kis mobil és mikrokijelzők (betűméretek és gombok csökkentése az "overflow" hibák elkerülésére).

Kódpélda: A részponzív Hamburger menü JavaScript logikája

JavaScript

// A navigáció mobilbarát lenyíló logikája

```
// HAMBURGER MENU TOGGLE
const hamburgerBtn = document.getElementById('hamburgerBtn');
const mainNav = document.getElementById('mainNav');

if (hamburgerBtn && mainNav) {
  hamburgerBtn.addEventListener('click', function(e) {
    e.stopPropagation();
    mainNav.classList.toggle('open');
  });
}
```

// Automatikus bezárás, ha a felhasználó a menün kívülre kattint

```
// Close menu when clicking outside
document.addEventListener('click', function(event) {
    if (mainNav.classList.contains('open') &&
        !mainNav.contains(event.target) &&
        !hamburgerBtn.contains(event.target)) {
        mainNav.classList.remove('open');
    }
});
```

2.6.3. Dinamikus Színkezelés és Kliensoldali Animációk

Az oldal egyik leginnovatívabb megoldása a "Csapatszínek" dinamikus kezelése. Mind a PHP backend (getTeamColor() függvény), mind a JavaScript tartalmaz egy színpalettát. A felhasználó által választott csapat színe CSS változók (pl. --team-color) segítségével fecskendeződik be a DOM-ba, így az egész oldal – a gomboktól kezdve a profilkártyák keretéig – alkalmazkodik a felhasználó preferenciájához.

A felhasználói élmény fokozása érdekében komplex CSS animációkat alkalmaztunk, amelyek hardveres gyorsítást (GPU acceleration) használnak:

- **cubic-bezier átmenetek:** A galériákban a kártyák nem lineárisan, hanem természetes "kiugró" és lassuló mozgással reagálnak az egérre (transition: all 0.4s cubic-bezier(0.2, 0.8, 0.2, 1)).
- **@keyframes effektek:** Az élő futam során a "Safety Car" és a "Live" jelzések pulzáló (blinker) hatást keltenek, a háttérben futó finom, piros dőlt vonalak (speed-lines) pedig a sebesség érzetét szimulálják.

2.6.4. Külső API integráció: A Pitwall Drag & Drop felülete

A "The Pitwall" tippjáték fejlesztése során egyetlen külső JavaScript könyvtárat, a **SortableJS**-t alkalmaztuk. Míg a mentést és a kiértékelést a backend végzi (JSON formátumban), a kliensoldali húzd-és-ejtsd élményt ez az optimalizált könyvtár biztosítja. A mobiltelefonokon történő véletlen átrendeződés elkerülése érdekében egy 200 ms-os késleltetést (delay) és 5 pixeles mozgási küszöböt (touchStartThreshold) konfiguráltunk a scriptben.

Kódpélda: A Pitwall interaktív tippelő felületének inicializálása

JavaScript

// Sortable.js inicializálása a pilóták átrendezéséhez

```
const sortable = new Sortable(listElement, {
  animation: 150,
  ghostClass: 'sortable-ghost',
  dragClass: 'sortable-drag',
  handle: '.driver-item',
  draggable: '.driver-item',
  scroll: true,
  scrollSensitivity: 30,
  scrollSpeed: 10,
  bubbleScroll: true,
  delay: 200,
  touchStartThreshold: 5,
  disabled: false,

  onEnd: function (evt) {
    document.body.style.overflow = '';
    document.body.classList.remove('sortable-dragging');
    evt.item.style.opacity = '';

    const dragHandles = listElement.querySelectorAll('.drag-handle');
    dragHandles.forEach(h => {
      h.style.color = '';
      h.style.opacity = '';
    });

    const badges = listElement.querySelectorAll('.position-badge');
    badges.forEach((badge, index) => {
      badge.innerHTML = (index + 1) + '.';
    });
  },
});
```

3. Felhasználói dokumentáció

Ez a dokumentáció a weboldal látogatóinak (vendégek) és regisztrált tagjainak szól. Célja, hogy lépésről lépésre bemutassa az F1 Fan Club felületének használatát, funkcióit, a navigáció logikáját, valamint segítséget nyújtson az esetlegesen felmerülő hibák és rendszerüzenetek értelmezésében.

3.1. A program célja és alapvető funkciói

Az F1 Fan Club egy Forma-1 rajongóknak készült, modern és interaktív weboldal. A felhasználó egy olyan komplex felületet kap a kezébe, ahol nemcsak a sportág múltbéli és jelenlegi statisztikáit böngészheti egy letisztult "Dark Mode" (sötét témájú) felületen, hanem aktív tagjává is válhat egy dinamikus közösségnek. A platform lehetőséget biztosít vélemények és képek megosztására (Paddock), valós idejű privát üzenetküldésre, élő versenyszimulációk követésére, valamint egy izgalmas, gamifikált tippjátékban („The Pitwall”) való részvételre is.

3.2. Szükséges hardvereszközök és szoftverek

Mivel a szoftver egy felhőalapú webes alkalmazás (Web Application), a használata nem igényel semmilyen speciális hardvert, és nem szükséges hozzá külső program telepítése sem.

- **Hardver:** Bármilyen internetképes eszköz (asztali számítógép, laptop, tablet vagy okostelefon). Az oldal teljes mértékben reszponzív (CSS Grid és Flexbox alapokon nyugszik), így a képernyő méretéhez és felbontásához automatikusan, minőségromlás nélkül igazodik.

- **Szoftver:** Egy modern, HTML5-öt és JavaScriptet támogató webböngésző (pl. Google Chrome, Mozilla Firefox, Safari, Microsoft Edge) és stabil internetkapcsolat.

3.3. Telepítés és indítás (Elérés)

A weboldal használatához semmilyen fájlt nem kell letölteni a felhasználó számítógépére. Az oldal a böngésző címsorába beírt `f1fanclub.hu` (vagy a fejlesztői szerverhez tartozó specifikus) URL címen érhető el. A szerver és a kliens közötti adatkommunikáció titkosított és biztonságos, amit az érvényes SSL tanúsítvány (HTTPS protokoll) garantál, így a felhasználók személyes adatai védve vannak a hálózati lehallgatástól.

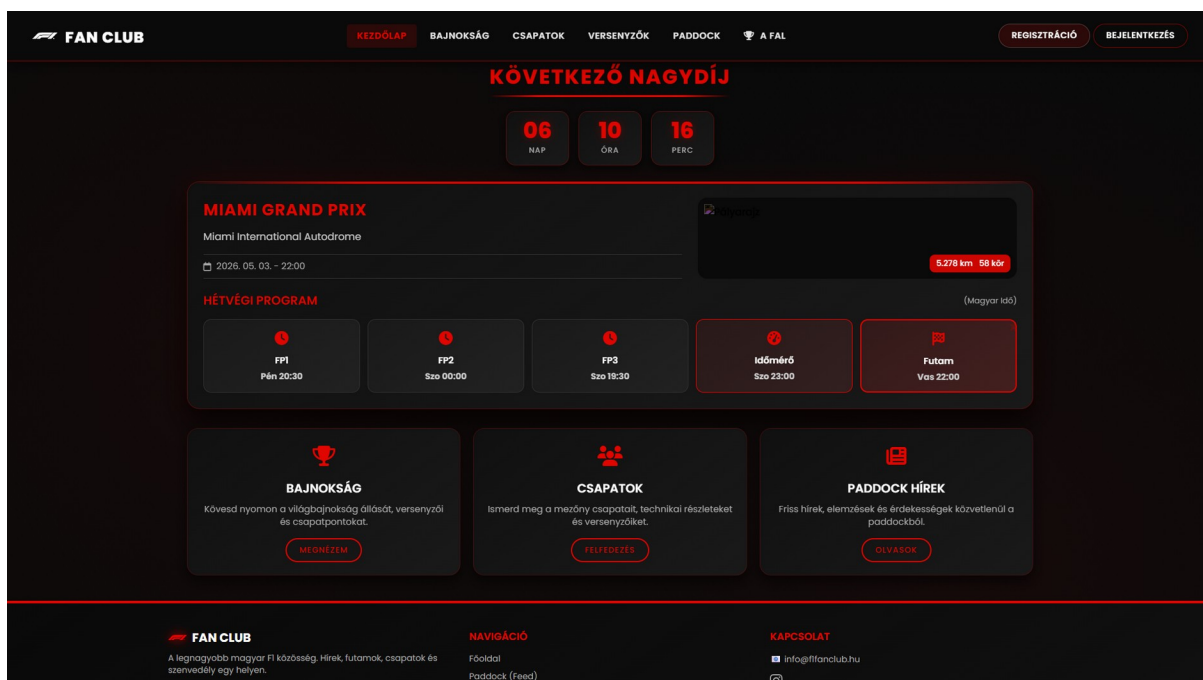
3.4. A program részletes bemutatása (Szabadon elérhető funkciók)

A weboldalra érkezve számos funkció azonnal, regisztráció nélkül is a látogatók (vendég felhasználók) rendelkezésére áll. Ezzel a megoldással a portál egyfajta "digitális névjegykártyaként" is funkcionál, amely betekintést enged a tartalomba, ezzel ösztönözve a látogatót a regisztrációra.

3.4.1. A Kezdőlap (`index.php`)

A főoldal a platform információs központja, amely a legfontosabb aktuális eseményeket gyűjti össze. A képernyő tetején egy látványos, szerveridőhöz szinkronizált dinamikus visszaszámláló modul kapott helyet. Ez másodpercre pontosan mutatja, hogy mennyi idő van hátra a következő F1-es versenyhétvégéig. A visszaszámláló alatt kártyás elrendezésben látható a teljes hétvégi program (Szabadedzések, Időmérő, Futam). A rendszer az adatbázisban tárolt időpontokat automatikusan a felhasználó helyi (magyar) idejére konvertálja, így elkerülhetők az időeltolódásból fakadó félreértések.

1.ábra: Az F1 Fan Club kezdőlapja a következő futam dinamikus visszaszámlálójával és a hétvégi menetrenddel:

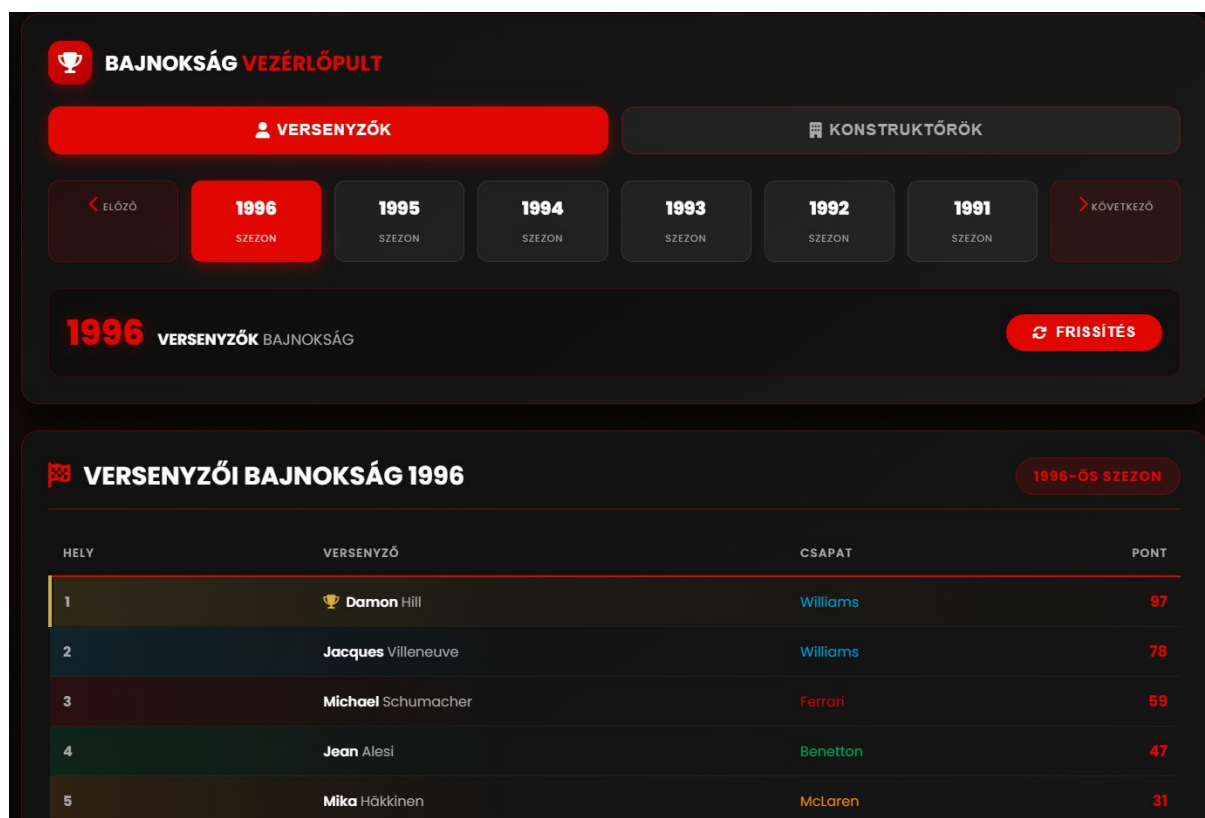


3.4.2. Bajnokság állása és Statisztikák (championship.php)

Ezen az aloldalon a felhasználók egy részletes, adatalapú felületet találnak. Az oldal lehetőséget ad arra, hogy egészen a sportág indulásáig, az 1950-es évekig visszamenőleg böngésszenek a történelmi adatok között. A felhasználó egy legördülő menüből választhatja ki a kívánt évet, majd a rendszer kilistázza az adott szezon konstruktóri és egyéni világbajnoki végeredményeit. A jelenleg futó szezon adatai automatikusan frissülnek a futamok leintése után. A felső navigációs sávból (vagy mobilnézet esetén a hamburger menüből) elérhetők a "Csapatok" és

"Versenyzők" menüpontok is, ahol vizuálisan gazdag adatlapokon ismerhetők meg a mezőny aktuális tagjai.

2. ábra: A történelmi és aktuális adatokat összesítő statisztikai felület vizuális megjelenése.



3.4.3. A Paddock – Hírfal (Vendég nézet)

A Paddock a weboldal központja, egy interaktív közösségi hírfolyam. Regisztráció nélkül a látogatók úgynevezett „olvasó” (Read-only) üzemmódban érik el ezt a felületet. Szabadon görgethetnek a tagok által közzétett posztok, feltöltött képek és vélemények között, láthatják, hogy melyik bejegyzés mennyi és milyen emoji reakciót kapott, de ők maguk nem szólhatnak hozzá és nem posztolhatnak. Ez a korlátozás

védi a közösséget a spam botoktól, és motiválja a valós érdeklődőket a fiók létrehozására.

3.4.4. Élő Verseny (Live Race) nézet

Versenyhétvégéken a látogatók beléphetnek a "Live" menüpontba, ahol valós időben (oldalfrissítés nélkül) követhetik a futam alakulását. A képernyőn látható a pilóták aktuális pozíciója, a köztük lévő időkülönbség, a gumikopás és az esetleges biztonsági autós (Safety Car) fázisok. A képernyő jobb oldalán (mobilon alul) futó rajongói élő chatet (Paddock Chat) a vendégek olvashatják, de üzenetet csak hitelesített fiókkal küldhetnek.

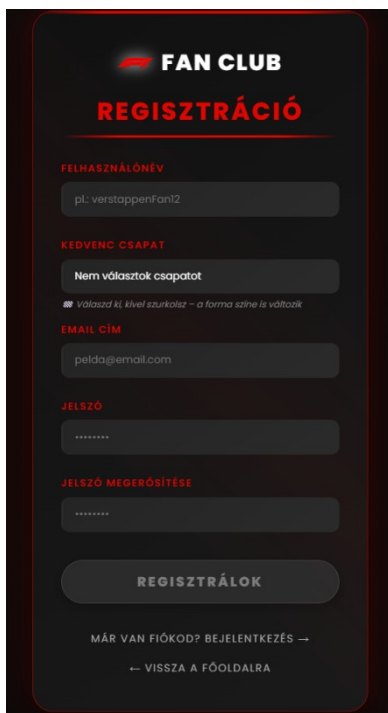
3.5. Regisztrációhoz kötött funkciók (A közösségi élmény)

A weboldal teljes potenciáljának kihasználásához egy rövid, de biztonságos regisztrációs folyamaton kell átesnie a felhasználónak.

3.5.1. Regisztráció és fiókaktiválás

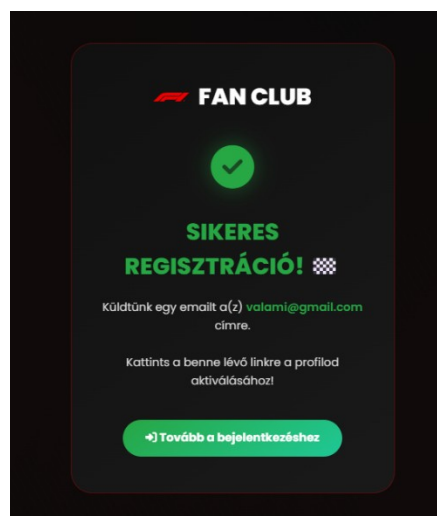
A regisztrációs űrlap vizuálisan reagál a felhasználó döntéseire: a kedvenc csapat kiválasztásakor az űrlap színei és dizájnelemei (pl. gombok, árnyékok) dinamikusan felveszik a kiválasztott istálló hivatalos arculati színeit. A rendszer megköveteli az erős jelszó használatát. Az űrlap elküldése után a fiók még inaktív; a véglegesítéshez a felhasználónak rá kell kattintania a megadott e-mail címre kiküldött, egyedi azonosítót tartalmazó aktiváló linkre (Double Opt-in védelem).

3.



The screenshot shows a registration form titled "FAN CLUB REGISZTRÁCIÓ". It includes fields for "FELHASZNÁLÓNÉV" (Username) with the example "pl. verstappenFan12", "KEDVENC CSAPAT" (Favorite Team) with a dropdown menu showing "Nem választok csapatot", "EMAIL CÍM" (Email Address) with the example "pelda@email.com", "JELSZÓ" (Password), and "JELSZÓ MEGERŐSÍTÉSE" (Confirm Password). At the bottom, there is a "REGISZTRÁLOK" button and a link "MÁR VAN FIÓKOD? BEJELENTKEZÉS →".

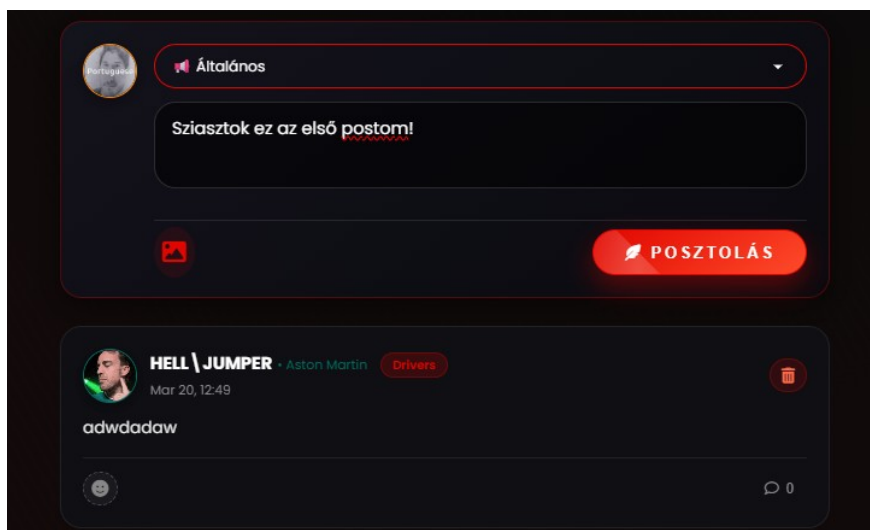
ábra: A regisztrációs felület, amely dinamikusan alkalmazkodik a kiválasztott F1-es csapat arculati színeihez.



3.5.2. Interakció a Paddockban

Sikeres bejelentkezés után a Paddock teljes értékűvé válik. A felhasználó a képernyő tetején található szövegdoboz segítségével hozhat létre új posztot. A rendszerezhetőség érdekében minden bejegyzéshez kötelező kiválasztani egy témakört (Kategória/Tag, pl. "Pletyka", "Technika", "Verseny"). Egy poszthoz maximum két darab képfájl csatolható (a rendszer csak JPG vagy PNG formátumot fogad el, maximum 5 MB méretig). A publikált posztokra a közösség tagjai különböző emoji-kkal reagálhatnak, és korlátlan számú szöveges hozzászólást írhatnak.

4. ábra: A Paddock közösségi felülete, bejelentkezett felhasználói (interaktív) nézetben.



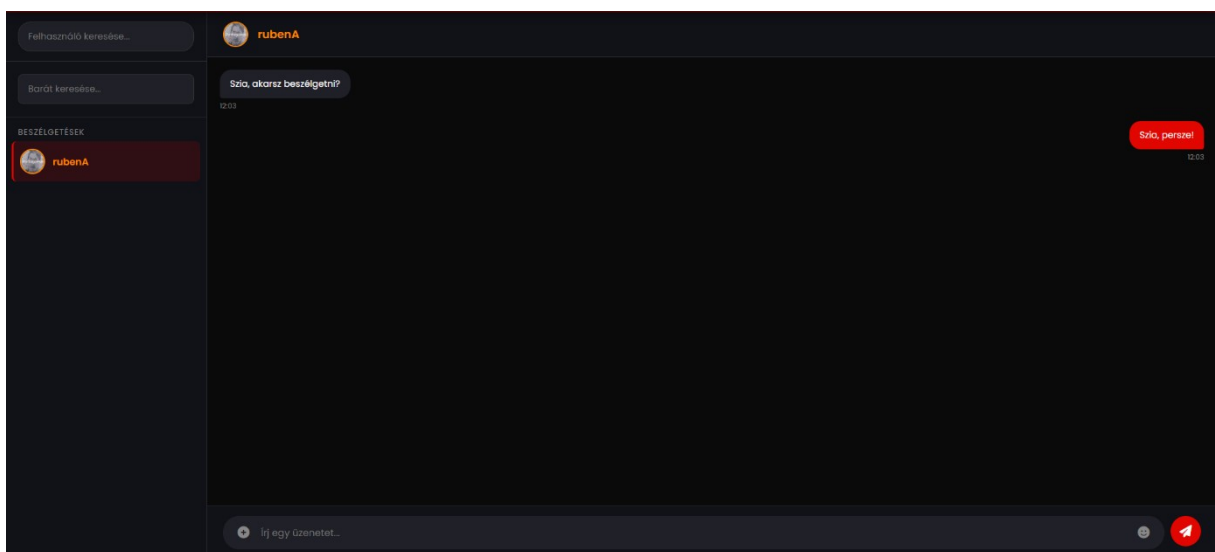
3.5.3. A Profilkezelő (Szuper Licenc)

A "Profilom" menüpont alatt a tagok egy saját, egyedi digitális "Szuper Licencet" találnak. Itt feltölthetnek egyéni profilképet (Avatart), megváltoztathatják a kedvenc csapatukat, és nyomon követhetik a tippjátékkal szerzett pontjaikat. Ezen a felületen van lehetőség az érzékeny adatok (név, e-mail cím, jelszó) módosítására is. A maximális biztonság érdekében a rendszer egyetlen adatmódosítást sem hajt végre azonnal: a felhasználó először egy jóváhagyó e-mailt kap, és a módosítás csak a levélben található link megerősítése után lép életbe.

3.5.4. Privát üzenetek (Üzenetközpont)

A tagok egy dedikált chat felületen tarthatják a kapcsolatot az ismerőseikkel. Új barátokat nagyon egyszerűen, a Paddockban a profilképükre kattintva, a felugró "Felhasználói kártyán" található "Barátnak jelölés" gombbal, vagy az üzenetek menüpont keresősávjának segítségével lehet felvenni. A rendszer vizuálisan is jelzi, ha a felhasználónak olvasatlan privát üzenete érkezett.

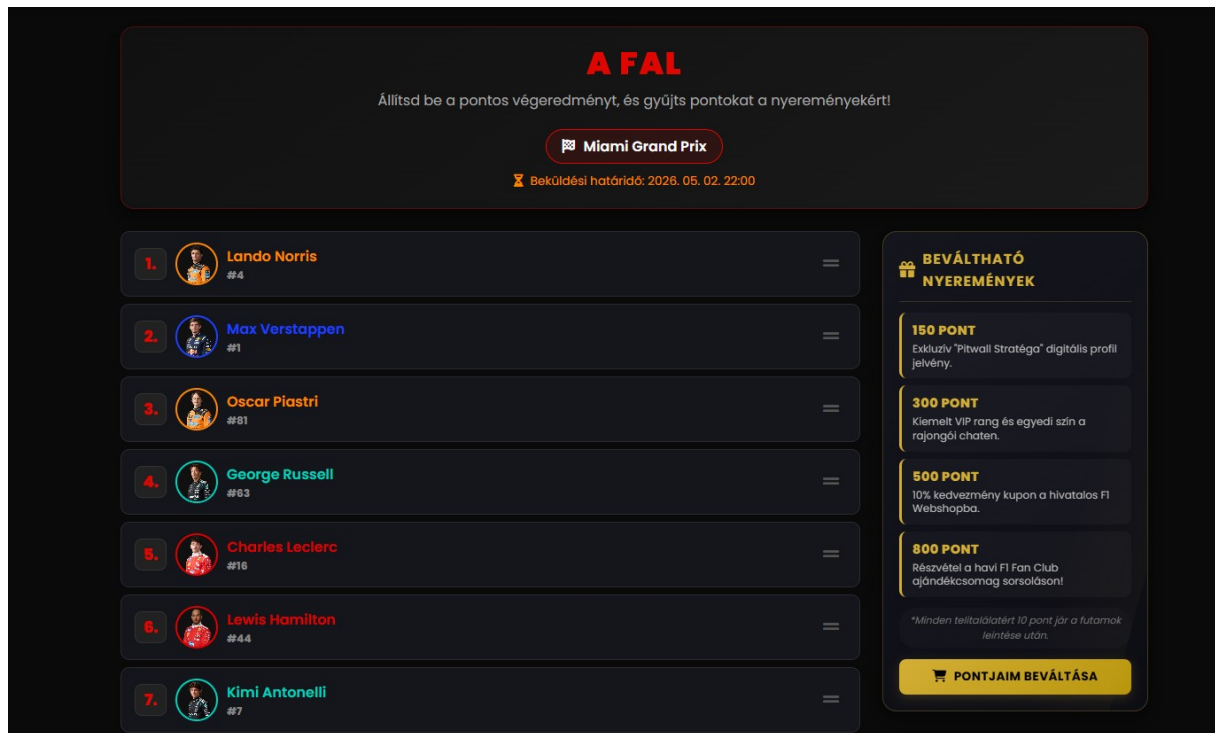
5. ábra: Privát üzenet felülete



3.5.5. „The Pitwall” – Az interaktív tippjáték

Az oldal leginnovatívabb közösségi funkciója a „The Pitwall” tippjáték. A felhasználók minden futam előtt megtippelhetik az első 20 pilóta beérkezési sorrendjét. A felület egy modern, csúszkás (Drag & Drop) módszerrel működik: a pilóták névjegyeit egyszerűen a kívánt pozícióra kell húzni (egérrel vagy érintőképernyőn ujjhúzással). A rendszer szigorú szabályokat alkalmaz: a beküldési határidő a futam rajtja előtt pontosan 24 órával lejár. Ekkor a felület inaktívvá (szürkévé) válik, a mentés gomb pedig eltűnik. A futam leintése után a szerver automatikusan összeveti a tippeket a valós végeredménnyel, és minden telitalálatért pontokat ír jóvá a felhasználó egyenlegén.

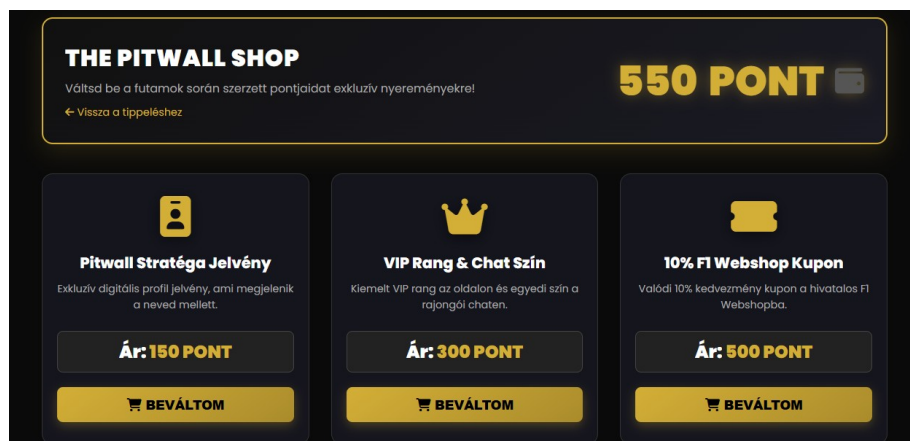
6. ábra: A "The Pitwall" tippjáték Drag & Drop mechanikát alkalmazó felülete.



3.5.6. A Pitwall Shop (Nyereményközpont)

A tippjátékkal szerzett virtuális pontokat a játékosok egy beépített, exkluzív "Webshop" felületen (Pénztárca) válthatják be. Az elérhető nyeremények között találhatók digitális jutalmak (pl. egyedi chat színek, VIP rangok, profilkiemelések), valamint fizikai ajándékokra váltható kuponok is (pl. 10% kedvezmény a hivatalos F1 áruházba). A beváltás gombra kattintva a rendszer azonnal levonja a pontokat az egyenlegről, és egy automatikus visszaigazoló e-mailt küld a további teendőkkel.

7. ábra: A Pitwall Shop, ahol a felhasználók a megszerzett pontjaikat válthatják be.



4. Összefoglalás

Amikor a projekttervezés legelső szakaszában megfogalmaztuk az *F1 Fan Club* alapkoncepcióját, egy rendkívül ambiciózus célt tűztünk ki magunk elé: egy olyan folyamatosan frissülő, interaktív rajongói oldalt akartunk létrehozni, amely túlmutat a hagyományos hírportálok keretein. Az eredeti vízió egy olyan modern, közösségépítő platform volt, amely a hivatalos Formula–1-es weboldal adatgazdagságát ötvözi a Red Bull interaktív megoldásaival és a Discord gördülékeny, valós idejű kommunikációs élményével. Visszatekintve az elmúlt hónapok munkájára, büszkén jelenthetjük ki, hogy ezt az átfogó, komplex víziót sikerült valósággá formálnunk.

Az ötlettől a publikusan elérhető, biztonságos és funkcionális weboldalig vezető út azonban rendkívül rögzös volt. A fejlesztés során számtalan, korábban sosem tapasztalt technikai akadályba és hibába ütköztünk. Szembesülnünk kellett azzal, hogy egy statikus weboldal megtervezése és egy valós idejű, több felhasználót egyidejűleg kiszolgáló webalkalmazás (Web Application) leprogramozása között hatalmas a szakadék. Rengeteg órát emésztett fel a kódok hibakeresése (debugging), a böngészők közötti kompatibilitási problémák elhárítása, és a reszponzív (mobilbarát) felületek CSS-anomáliáinak javítása. Különösen nagy kihívást jelentett a szerveroldali aszinkron folyamatok (AJAX Polling) összehangolása úgy, hogy az élő chat és a telemetria ne terhelje túl az adatbázist. Volt, hogy egyetlen elért SQL lekérdezés vagy egy rosszul kezelt JavaScript Promise miatt az egész rendszer összeomlott.

Ugyanakkor pontosan ezek a nehézségek, a megszámlálhatatlan hibaüzenet és az azokra keresett megoldások eredményezték a legnagyobb szakmai fejlődésünket. A vizsgaremek elkészítése során a tudásunk a sokszorosára nőtt: a korábbi, felületes programozási ismereteink egy mély, strukturált rendszerszemléletté alakultak át. Megtanultuk a gyakorlatban alkalmazni a relációs adatbázisok normalizálását, az aszinkron kliens-szerver kommunikációt (Fetch API, JSON), és olyan modern kiberbiztonsági eljárásokat implementáltunk (jelszó-hashelés, Prepared Statements, Double Opt-in), amelyek a valódi munkaerőpiacon is alapkövetelménynek számítanak. A „The Pitwall” tippjátéknál alkalmazott JSON alapú adattárolás, vagy a chat optimalizált architektúrája mind olyan mérnöki döntések voltak, amikre kifejezetten büszkék vagyunk.

Minden nehézség ellenére egy rendkívül korrekt, stabil és biztonságos rendszert sikerült összehoznunk, amely maradéktalanul teljesíti az eredeti célkitűzéseket. Létrehoztunk egy platformot, ahol a fiatal és felnőtt rajongók nemcsak híreket és statisztikákat olvashatnak, hanem egy interaktív, gamifikált közösség tagjai is lehetnek.

Fontos hangsúlyozni, hogy az *F1 Fan Club* jelenlegi állapota számunkra nem a végállomást, hanem egy stabil, jól skálázható alapot jelent. A jövőben is szeretnénk folytatni az oldal fejlesztését: célunk a Formula 2-es adatok integrálása, a valós élő videós közvetítések (streamek) beágyazása, és egy dedikált hírszerkesztői modul visszavezetése. Ez a projekt bebizonyította számunkra, hogy kellő kitartással, kutatómunkával és csapatmunkával képesek vagyunk egy nulláról felépített, modern szoftveripari elvárásoknak megfelelő IT projektet a tervezőasztaltól egészen a sikeres publikálásig végigvinni.

Technikai továbbfejlesztési lehetőségek

Bár az *F1 Fan Club* a jelenlegi állapotában egy stabil, funkciókban gazdag, és éles szerverre telepíthető vizsgaremek, a szoftverfejlesztés egy véget nem érő folyamat. A rendszer architektúráját úgy alkottuk meg, hogy a jövőben modulárisan bővíthető legyen. Fejlesztőcsapatunk az alábbi technikai mérföldköveket jelölte ki a jövőbeli verziókhoz (Roadmap):

1. **PWA (Progressive Web App) integráció:** Tervben van a weboldal telepíthetővé tétele mobil eszközökre Service Worker-ek segítségével. Ez lehetővé tenné a statikus adatok (versenynaptár) offline elérését, és drasztikusan gyorsítaná az oldalbetöltést a gyorsítótárzás révén.
2. **WebSocket kommunikáció:** A jelenlegi Short Polling (3 másodpercenkénti AJAX kérés) technológiát a jövőben WebSocket kapcsolatra cserélnénk. Ezzel valódi, kétirányú, késleltetés nélküli kapcsolat jöhetne létre az élő chat és a telemetria moduloknál, nagyságrendekkel csökkentve a szerver HTTP kérésekből fakadó terhelését.
3. **Harmadik féltől származó hitelesítés (OAuth 2.0):** A felhasználói bázis növelése érdekében integrálnánk a Google, Facebook vagy Discord API-kat, lehetővé téve az egykattintásos (Single Sign-On) regisztrációt és belépést.

4. **Automatizált feladatok (Cron Jobs):** Jelenleg a futameredmények és az élő szimulációs adatok adminisztrátori beavatkozást igényelnek. A jövőben szerveroldali időzített feladatokkal (Cron) a rendszer automatikusan képes lenne szinkronizálni a statisztikákat a globális Ergast/Jolpi F1 API-ból.
 5. **Modern Frontend keretrendszer bevezetése:** Ahogy a projekt mérete és a DOM manipulációk száma növekszik, indokoltá válhat a natív JavaScript kód átírása egy komponensalapú keretrendszerre (pl. React.js vagy Vue.js) a még gördülékenyebb (Single Page Application) élmény eléréséhez.
-

5. Irodalomjegyzék és felhasznált források

A rendszer tervezése és fejlesztése során a legfrissebb hivatalos dokumentációkra, valamint elismert szakmai fórumokra támaszkodtunk.

Hivatalos technikai dokumentációk:

1. **PHP Official Documentation:** A szerveroldali logika, a session-kezelés és a password_hash() kriptográfiai függvények megvalósításához. (Elérhető: <https://www.php.net/docs.php>)
2. **MySQL / MariaDB Reference Manual:** A relációs adatbázis tervezéséhez, a JOIN kapcsolatok és a Prepared Statements (bind_param) felépítéséhez. (Elérhető: <https://dev.mysql.com/doc/>)
3. **Oktatási és fejlesztői platformok: W3Schools:** Szintaktikai alapok, CSS animációk és gyors HTML referenciák. (Elérhető: <https://www.w3schools.com>)
4. **Stack Overflow:** Az egyedi technikai kihívások, hibakeresési (debugging) folyamatok, valamint a szerveroldali gyorsítótár-tiltás (Cache-Control) beállításainak kutatása. (Elérhető: <https://stackoverflow.com>)
5. **Ergast Developer API / Jolpi:** A Formula 1-es történelmi adatok, futameredmények és versenynaptárak strukturált lekéréséhez. (Elérhető: <http://ergast.com/mrd/>)

6. **Sortable.js Documentation:** A tippjáték („The Pitwall”) Drag & Drop (húzd és ejtsd) mechanikájának integrálásához. (Elérhető:

<https://sortablejs.github.io/Sortable/>)

7. **Font Awesome:** A weboldalon használt reszponzív, vektoros ikonok (SVG) gyűjteménye. (Elérhető: <https://fontawesome.com/>)

8. **Google Fonts:** A teljes felületen alkalmazott „Poppins” betűkészlet licencelése és integrációja. (Elérhető: <https://fonts.google.com/>)

Szakmai és dizájn inspirációk:

9. **Formula 1 Official Media Guide:** A hivatalos csapatnevek, technikai specifikációk és a csapatok arculati színekódjainak (Hex) hitelesítéséhez. (Elérhető: <https://www.formula1.com/>)

10. **Red Bull Racing & Discord:** Az interaktív gamifikációs elemek és a sötét témájú, letisztult közösségi chat-felületek (UI/UX) vizuális koncepciójának kialakításához.